CUNEX Voting

TECHNICAL DESIGN DOCUMENT

High-level Architecture

ระบบสร้างและตอบโพลสำหรับนิสิตจุฬาลงกรณ์มหาวิทยาลัย บนพื้นฐาน CUNEX

เวอร์ชัน	วันที่	สถานะ	Stack
Draft v0.1	4 ต.ค. 2569	รอทีมรีวิว	NestJS · Postgres

สรุปใน 1 นาที

1 **Modular monolith**
NestJS codebase เดียว แยกรันเป็น API กับ Worker · Postgres เป็นข้อมูลหลัก · Redis เก็บ session, cache และ queue

2 **ยืนยันตัวตนผ่าน CUNEX**
แลก token ที่ `/profile` ได้คณะและปีเข้าสำหรับกำหนดขอบเขตผู้ตอบ ผู้ใช้ Google/Email ตอบได้เฉพาะโพลสาธารณะ

3 **เจ้าของโพลไม่รู้ว่าใครตอบอะไร**
ballot ไม่มี user_id มีแค่ HMAC key ซึ่งยังกันโหวตซ้ำและแก้คำตอบได้

4 **Cloud-agnostic**
ใช้แค่ container, Postgres, Redis และ S3 ย้ายไป cloud ใดหรือ on-prem ของจุฬาฯ ก็ได้

5 **รองรับช่วงคนเข้าพร้อมกัน**
โหวตประมาณ 200 req/s ดูผลมากกว่า 1,000 req/s (ตอบจาก cache) ไม่ต้องใช้ microservices

6 **ต้องแก้ดีไซน์ 2 จุด**
เปลี่ยน "สาขา" เป็น "ปีเข้า" · ป๊อปอัปบัญชีทดลองเปิดได้เฉพาะ dev/staging

01 บริบทและขอบเขต
02 ข้อตัดสินใจ
03 Architecture
04 Components
05 Trade-offs
06 Flow การทำงาน
07 Privacy
08 เมื่อระบบล่ม
09 Data model
10 คำถามที่ยังเปิด

01 บริบทและขอบเขต

ระบบนี้ทำอะไร ใหญ่แค่ไหน และอะไรอยู่นอก scope

เว็บแอปให้นิสิตสร้างโพล ส่งให้ Admin อนุมัติ แล้วเผยแพร่ให้นิสิตตอบ เปิดได้ 2 ทาง คือจาก **เมนู ลิงก์ หรือ notification ใน CUNEX** และเปิดเว็บโดยตรง

หน้าจอ
เข้าสู่ระบบ
บัญชีจุฬาฯ (CUNEX) · Google · Email พร้อมข้อความ "ระบบจะไม่แสดงชื่อผู้ตอบให้เจ้าของโพลเห็น"

หน้าจอ
สร้างโพล
ขอบเขตผู้ตอบ · ชื่อ คำอธิบาย เวลาปิด รูปปก · element 9 ชนิด เรียงลำดับได้ · บันทึกร่าง / ส่งขออนุมัติ

หน้าจอ
ตอบโพล
ตอบทีละหน้า มี progress bar · radio / checkbox พร้อมคำอธิบาย · Preview mode สำหรับเจ้าของ

ตั้งค่า
ตัวเลือกของโพล
แสดงผลแบบเรียลไทม์ · ลงคะแนนซ้ำ (= แก้คำตอบ) · เผยแพร่ในหน้ารวม

Element ที่ต้องรองรับ

คำถาม · 4 ชนิด
ข้อความ · เลือกหนึ่งข้อ · เลือกหลายข้อ · เลือกรายการ
ตัวเลือกแนบรูปและป๊อปอัปข้อมูลได้

องค์ประกอบ · 5 ชนิด
คำอธิบาย · รูปภาพ (+คำบรรยาย) · ป๊อปอัปข้อมูล
หัวข้อพับเก็บ · ขึ้นหน้าใหม่

ขนาดระบบที่ออกแบบไว้

~40k
นิสิตทั้งหมด (ค่าประมาณ)

~200 req/s
โหวตช่วงพีค เมื่อส่ง push ทั้งมหาวิทยาลัย (30% ตอบใน 10 นาที)

~1,250 req/s
ดูผลพร้อมกัน 5,000 คน poll ทุก 4 วินาที (ตอบจาก cache)

ปริมาณโหลดไม่ใช่โจทย์หลัก โจทย์หลักคือผลโหวตต้องถูกต้อง ผู้ตอบต้องไม่ถูกระบุตัวตน ต้องสอดคล้องกับ PDPA และทีมต้องดูแลระบบระยะยาวได้

นอก scope

การเลือกตั้งอย่างเป็นทางการ (สภานิสิต / อบจ.) ซึ่งต้องการ verifiability และ coercion-resistance ระดับที่สูงกว่านี้มาก

02 ข้อตัดสินใจที่ตกลงแล้ว

12 ข้อที่เป็นฐานของ architecture ทั้งหมด

รูปแบบระบบ
Modular monolith
โหลดต่ำ ทีมเล็ก ใช้ transaction เดียวได้

ภาษา / framework
NestJS (TypeScript)
validate element หลายชนิดได้ง่าย แชร์ type กับ frontend ได้

ที่ deploy
Cloud-agnostic
ยังไม่รู้ข้อกำหนดเรื่อง data residency

ยืนยันตัวตนหลัก
CUNEX LLE (token → /profile)
ได้คณะและปีเข้าที่เชื่อถือได้

ผู้ใช้ Google / Email
ตอบได้เฉพาะโพลสาธารณะ
ไม่มีข้อมูลคณะให้ยืนยัน

ขอบเขตผู้ตอบ
คณะ + ปีเข้า
CUNEX ไม่ส่งข้อมูลสาขา ต้องแก้ดีไซน์

สร้างโพล / Admin
นิสิตทุกคน / allowlist ในระบบเรา
CUNEX ไม่มีข้อมูล role ให้

"ลงคะแนนซ้ำ"
= แก้คำตอบได้
จึงต้องใช้ pseudonymous key (HMAC)

แก้โพลหลังอนุมัติ
ล็อกโครงสร้าง แก้ได้แค่เวลาปิด
ผลไม่เพี้ยน ไม่ต้องทำ versioning

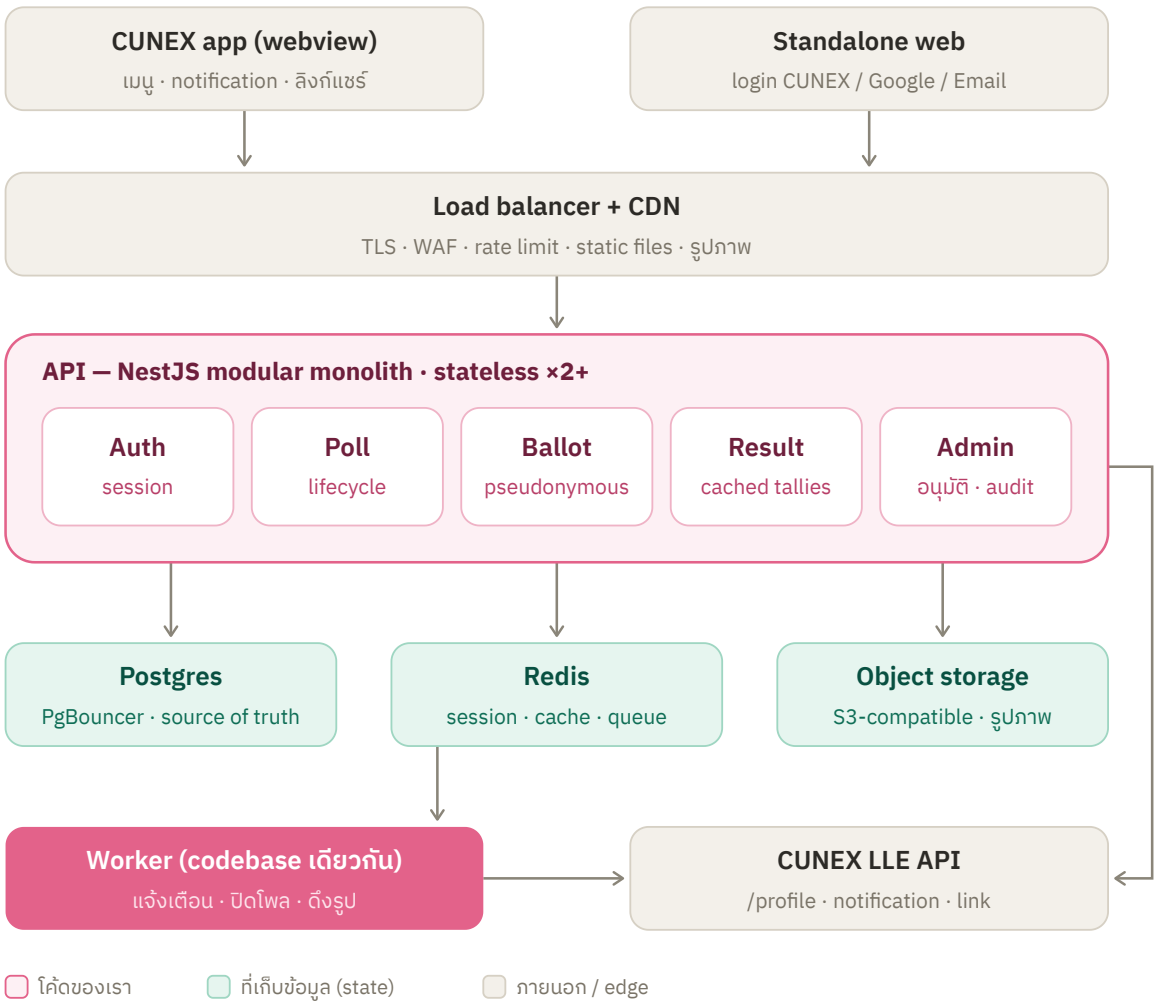
ปิดแสดงผลเรียลไทม์
ผู้ตอบเห็นผลหลังปิดโพล
เจ้าของโพลเห็นผลได้ตลอด

คำตอบที่ยังไม่ครบ
ไม่เก็บ ส่งครั้งเดียวตอนจบ
ง่าย และเก็บข้อมูลน้อยที่สุด

Notification
CUNEX notification ผ่าน queue + worker
มีโควตา ต้องส่งเป็น batch และมี retry

03 Architecture overview

ทุก component ในภาพเดียว



รูปที่ 1 High-level architecture

อ่านภาพนี้อย่างไร

Client → Edge → API

เข้าได้ 2 ทาง แต่ทุก request ผ่าน Load balancer + CDN เข้าสู่ API ชุดเดียวกัน

API แบบ stateless

process เดียวที่แบ่ง module ภายในชัดเจน รันอย่างน้อย 2 ตัว เพราะไม่เก็บ state ในตัวเอง

Worker

ใช้โค้ดชุดเดียวกับ API แต่รันแยก process รับงานช้าจาก queue เช่นแจ้งเตือน ปิดโพล ดึงรูป

Source of truth

Postgres เป็นแหล่งข้อมูลจริงเพียงแห่งเดียว Redis เก็บเฉพาะข้อมูลที่หายได้หรือสร้างใหม่ได้

04 หน้าทีของแต่ละ component

แต่ละกล่องในรูปที่ 1 ทำอะไร

Component	หน้าที่
EDGE **Load balancer + CDN**	TLS, WAF, rate limit ต่อ IP · เสิร์ฟไฟล์ frontend และรูปจาก object storage · กระจาย request ไปยัง API หลาย replica
API **Auth**	แลก CUNEX token ที่ `/profile` · Google OIDC · Email OTP · สร้างและลบ session ใน Redis · ตรวจ admin allowlist
API **Poll**	CRUD โพลและ element 9 ชนิด (validate `config` ด้วย zod) · state machine ของโพล · ออก presigned URL สำหรับอัปโหลดรูป
API **Ballot**	ตรวจสิทธิ์และเวลา · validate คำตอบ · คำนวณ `voter_key` · upsert ballot ใน transaction เดียว
API **Result**	ตรวจสิทธิ์การดูผล · อ่าน cache หรือ query `GROUP BY` · ซ่อนกลุ่มที่มีคนตอบน้อยกว่า k คน
API **Admin**	คิวโพลรออนุมัติ · อนุมัติหรือตีกลับพร้อมเหตุผล · จัดการ admin · บันทึก audit log
WORKER **Background jobs**	ส่ง CUNEX notification เป็น batch ตามโควตา · สร้าง UniversalLink · ปิดโพลเมื่อถึง `closes_at` · ดึงรูปจาก URL ภายนอก · retry
STATE **Postgres + PgBouncer**	ข้อมูลทั้งหมดที่หายไม่ได้ · trigger ล็อกโครงสร้างโพล · PgBouncer กันจำนวน connection พุ่งเมื่อ autoscale
STATE **Redis**	session และ OTP (TTL) · cache ผลโหวต 3 วินาที · queue ของ worker (BullMQ) · ตัวนับ rate limit
STATE **Object storage**	รูปปกและรูปของ element/ตัวเลือก · S3 API ใช้ได้ทั้ง AWS, GCS และ MinIO แบบ on-prem
EXTERNAL **CUNEX LLE**	`/profile` · `/notification` · `/create_link` · `/add_board` (อาจใช้ในอนาคต)

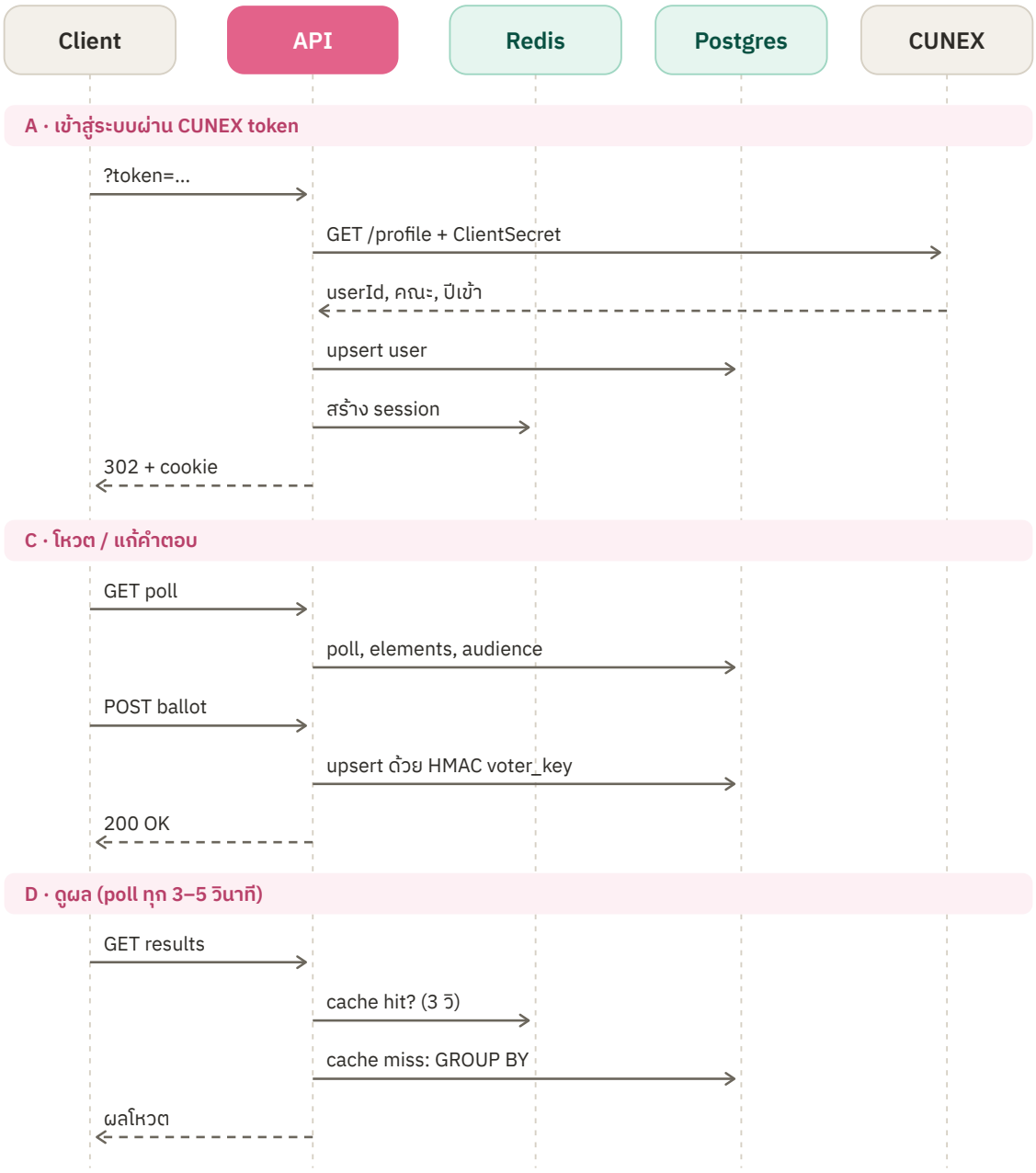
05 Trade-off และเหตุผล

เลือกอะไร ไม่เลือกอะไร และทำไม

เรื่อง	เลือก	ไม่เลือก	เหตุผล
รูปแบบระบบ	**Modular monolith**	~~Microservices~~	ที่ 200 req/s microservices ทำให้ต้องจัดการ distributed transaction และ ops เพิ่มโดยไม่ได้ประโยชน์ ถ้าแบ่ง module ดีตั้งแต่แรก วันหลังก็แยกออกได้
แพลตฟอร์ม	**Custom backend + Postgres**	~~Supabase / BaaS~~	logic สำคัญจะไปอยู่ใน RLS และ SQL function ซึ่งทดสอบยาก region ใกล้สุดคือสิงคโปร์ และผูกกับ vendor
ภาษา	**NestJS**	~~Go~~	ทั้งสองรับโหลดนี้ไหว จึงตัดสินที่ productivity: element 9 ชนิดใช้ zod discriminated union ได้ตรงๆ และแชร์ type กับ frontend ได้
Deploy	**Container + autoscale (ขั้นต่ำ 2)**	~~Serverless~~	serverless มี cold start และ connection ไปยัง DB พุ่งตอน spike ส่วน container คาดเดาพฤติกรรมได้ง่ายกว่าและย้ายที่ deploy ได้
Database	**Postgres + JSONB**	~~Document DB~~	config ของ element ต่างกันจึงเก็บเป็น JSONB แต่ผลต้อง aggregate ตามคณะ จึงต้องมี SQL และ foreign key
Session	**httpOnly cookie + Redis**	~~JWT~~	เพิกถอนได้ทันที (เช่น ถอดสิทธิ์ admin) และโหลดระดับนี้ไม่ต้องการ stateless JWT
นับผล	**GROUP BY + cache 3 วินาที**	~~Counter / INCR~~	การแก้คำตอบต้องลดคะแนนตัวเลือกเดิมด้วย ซึ่ง counter ทำให้ตัวเลขเพี้ยนง่ายและเกิด hot-row lock ส่วน GROUP BY ได้ผลถูกต้องเสมอ เร็วระดับมิลลิวินาที
Realtime	**Polling 3–5 วินาที**	~~WebSocket / SSE~~	ทำงานใน webview ได้แน่นอน และไม่ต้องตั้ง sticky session ที่ LB ภายหลังเปลี่ยนเป็น SSE ได้ถ้าต้องการ
รูปจาก URL	**Worker ดึงมาเก็บเอง**	~~Hotlink~~	hotlink ทำให้เจ้าของ server รูปเห็น IP และเวลาของคนเปิดโพล ซึ่งทำลายการไม่ระบุตัวตน และรูปอาจหายภายหลัง
งานช้า	**Queue + Worker**	~~ทำใน request~~	CUNEX notification มีโควตาและอาจล้มเหลว ไม่ควรทำให้ API ช้าลงหรือล่มตาม

06 Flow การทำงาน

5 เส้นทางหลักของระบบ ตั้งแต่ login จนปิดโพล



รูปที่ 2 Sequence ของเส้นทางที่ผู้ใช้เจอบ่อยที่สุด · เส้นทึบ = request · เส้นประ = response

A เข้าใช้งาน

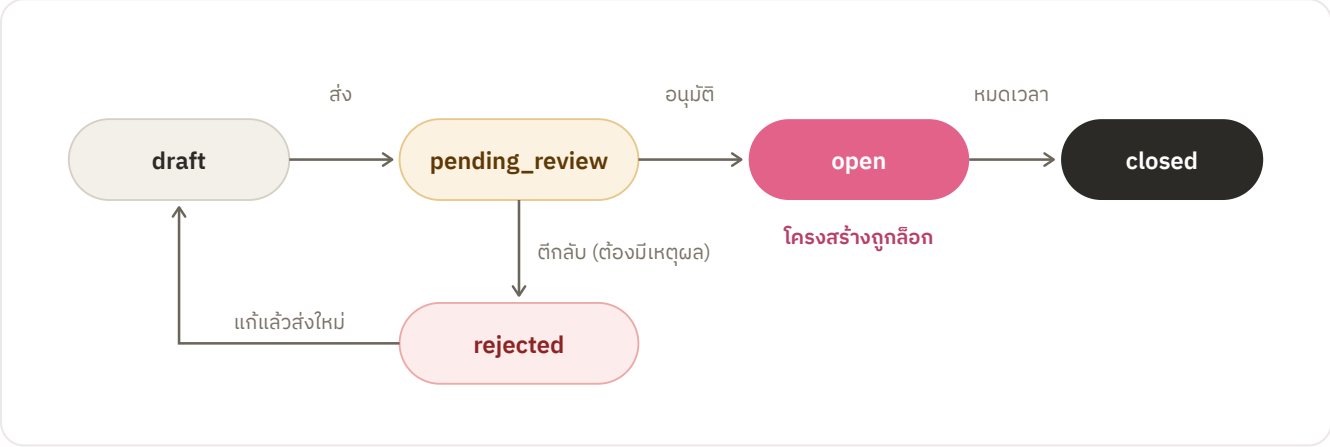
เข้าจาก CUNEX
เมนู, notification หรือลิงก์แชร์ CUNEX เปิดเว็บเราพร้อม `?token=`

เปิดเว็บตรง
กด "เข้าสู่ระบบด้วยบัญชีจุฬาฯ" → login ที่ CUNEX → กลับมาพร้อม `?token=`

Google / Email
ได้ user ที่ `is_chula_verified = false` ตอบได้เฉพาะโพลสาธารณะ

API เอา token ไปแลกที่ `/profile` (แนบ `ClientSecret` ฝั่ง server) → บันทึกหรืออัปเดต user และคณะ → สร้าง session → ตอบ 302 พร้อม httpOnly cookie ไปยัง URL ที่**ไม่มี token** ติดอยู่ หลังจากนี้ไม่ต้องติดต่อ CUNEX อีก

B สร้างโพล → อนุมัติ → เปิด



รูปที่ 3 Poll lifecycle · หลัง `open` แล้ว trigger ใน DB ไม่ยอมให้แก้คำถามหรือตัวเลือก

1. **สร้างร่าง**
   บันทึกโพล, element และตัวเลือก สถานะเป็น `draft`
2. **รูปภาพ**
   ถ้าอัปโหลดไฟล์ client จะอัปโหลดตรงไป object storage ด้วย presigned URL ถ้าใส่ URL รูป worker จะดึงรูปมาเก็บไว้ในระบบเรา
3. **ส่งขออนุมัติ**
   สถานะเปลี่ยนเป็น `pending_review` และขึ้นในคิวของ Admin
4. **Admin ตัดสิน**
   บันทึกผลใน `poll_reviews` และ `audit_logs` ถ้าตีกลับ สถานะเป็น `rejected` แก้แล้วส่งใหม่ได้
5. **เปิดโพล**
   โครงสร้างถูกล็อก แล้วส่งงาน 3 อย่างเข้า queue: สร้าง UniversalLink · ส่ง notification ให้ user ที่อยู่ในขอบเขตเป็น batch ตามโควตา · ตั้งงานปิดโพลที่เวลา `closes_at`

C โหวต / แก้คำตอบ

1. **เช็กเวลา**
   สถานะต้องเป็น `open` **และเวลาปัจจุบันยังไม่เลย** `closes_at` ต้องเช็กเวลาด้วย เพราะ worker อาจปิดโพลช้ากว่ากำหนด
2. **เช็กสิทธิ์**
   คณะและปีเข้าของผู้ใช้ต้องตรงกับ `poll_audience_rules`
3. **เช็กคำตอบ**
   ตอบข้อบังคับครบ ตัวเลือกเป็นของคำถามนั้นจริง จำนวนที่เลือกไม่เกินกำหนด
4. **สร้าง voter_key**
   คำนวณ `HMAC(pepper, poll_id‖user_id)` ในหน่วยความจำ แล้วทิ้ง user_id ไม่ส่งลง DB
5. **Upsert**
   บันทึกใน transaction เดียว: ยังไม่เคยโหวตจะสร้างใหม่ · เคยโหวตและเปิดให้แก้จะแทนที่คำตอบเดิม · เคยโหวตแต่ไม่เปิดให้แก้จะตอบ `409`

> ระหว่างที่ผู้ใช้กด "ถัดไป" ทีละหน้า คำตอบอยู่ฝั่ง client ทั้งหมด จะส่งมาครั้งเดียวตอนกดส่ง

D ดูผล

สิทธิ์
ใครเห็นผลเมื่อไหร่
เจ้าของโพลเห็นได้ตลอด · ผู้ตอบเห็นได้ทันทีถ้าเปิดแสดงผลเรียลไทม์ ถ้าไม่เปิดต้องรอโพลปิด

Cache
Redis 3 วินาที
ถ้าไม่มีใน cache ค่อยรัน `GROUP BY` และคนที่ขอพร้อมกันรอผลจาก query เดียวกัน ผลคือ 5,000 คน ทำให้ DB ถูก query แค่ประมาณ 1 ครั้งทุก 3 วินาที

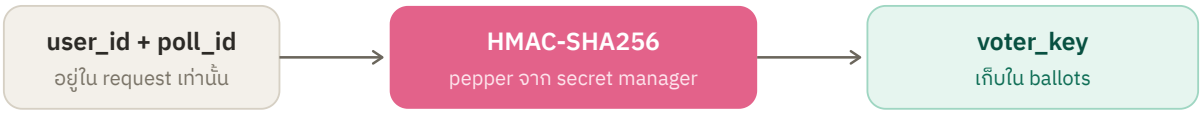
Privacy
k-anonymity
ผลแยกตามคณะหรือปี ถ้ากลุ่มไหนมีคนตอบน้อยกว่า k คนจะไม่แสดง

E ปิดโพล

ถึงเวลา `closes_at` worker จะเปลี่ยนสถานะเป็น `closed` และแจ้งผลได้ถ้าต้องการ ถ้า worker ทำงานช้า ขั้นตอน C.1 ยังกันโหวตหลังหมดเวลาได้ เจ้าของกดปิดโพลก่อนกำหนดได้

07 การไม่ระบุตัวตนและข้อมูลส่วนบุคคล

ซ่อนตัวตนจากเจ้าของโพล แต่ยังกันโหวตซ้ำและแก้คำตอบได้



- หา ballot เดิมเจอ → แก้คำตอบได้
- UNIQUE(poll_id, voter_key) → โหวตซ้ำไม่ได้
- ใน DB ไม่มี user_id → แค่ได้ DB ไปก็ย้อนหาตัวคนไม่ได้
- key ต่างกันทุกโพล → เชื่อมข้ามโพลไม่ได้

รูปที่ 4 วิธีสร้าง voter_key

มาตรการ	กันอะไร
ballot **ไม่มี user_id** มีเฉพาะ `voter_key`	แค่ได้ข้อมูลจาก DB อย่างเดียวจะรู้ไม่ได้ว่าใครตอบอะไร
`pepper` อยู่ใน secret manager ไม่ได้อยู่ใน DB	DB backup ที่รั่วออกไปเอามาย้อนหาตัวตนไม่ได้
เวลาโหวตปัดเป็นนาที	เอาเวลาไปเทียบกับ log การ login แล้วเดาตัวคนไม่ได้
k-anonymity ตอนแสดงผลแยกกลุ่ม	กลุ่มที่มีคนตอบ 2–3 คนเดาตัวตนไม่ได้
ห้ามบันทึกการโหวตใน audit log และ app log	log ไม่กลายเป็นช่องทางเชื่อมคนกับคำตอบ
ดึงรูปภายนอกมาเก็บเอง ไม่ hotlink	server รูปภายนอกไม่เห็น IP ของผู้ที่เปิดโพล
ไม่เก็บ `studentId` เก็บแค่คณะและปีเข้า	ลดปริมาณข้อมูลส่วนบุคคลที่เก็บ ตามหลัก PDPA
Token ของ CUNEX ใช้ครั้งเดียว แล้ว redirect ไป URL ที่ไม่มี token	token ไม่รั่วผ่าน Referer หรือ history

ข้อจำกัดที่ทีมควรรู้: ผู้ที่ถือทั้ง DB และ pepper ย้อนหาตัวตนได้ (ไล่ลอง userId ราว 4 หมื่นค่า) ระดับนี้เพียงพอสำหรับข้อกำหนด "ซ่อนจากเจ้าของโพล" แต่ไม่เพียงพอสำหรับการเลือกตั้งอย่างเป็นทางการ

08 เมื่อแต่ละส่วนล่ม

ผลกระทบและการรับมือ เรียงจากเบาไปหนัก

ส่วนที่ล่ม	ระดับ	ผลกระทบ	การรับมือ
API 1 replica	ไม่กระทบ	ผู้ใช้ไม่รู้สึกถึงความต่าง	ขั้นต่ำ 2 replica · health check ที่ LB
Worker	ล่าช้า	notification และการปิดโพลอัตโนมัติล่าช้า · **การโหวตยังทำงานและยังถูกต้อง**	API เช็ก `closes_at` เอง · งานรออยู่ใน queue
CUNEX	บางส่วน	login ใหม่ไม่ได้ · notification ค้างใน queue · **คนที่ login แล้วยังโหวตได้**	session ของเราไม่ขึ้นกับ CUNEX · worker retry
Traffic พุ่ง	ช้าลง	latency สูงขึ้น	autoscale · PgBouncer · rate limit · load test ล่วงหน้า
Redis	ทุกคนออกจากระบบ	session หาย ทุกคนต้อง login ใหม่ · DB รับโหลดเพิ่มชั่วคราว · **ผลโหวตไม่หาย**	managed Redis ที่มี replica · ข้อมูลจริงอยู่ใน Postgres
Postgres	ระบบหยุด	ใช้งานไม่ได้ทั้งระบบ	managed DB + standby + PITR backup

✓ **ก่อนเปิดใช้งานจริง:** load test สถานการณ์ "ส่ง push ทั้งมหาวิทยาลัย" คือคนเข้ามาโหวตพร้อมกันและเปิดหน้าผลพร้อมกันในเวลาไม่กี่นาที

09

Data model โดยสรุป

14 ตาราง 5 กลุ่ม · รายละเอียดเต็มที่ `db/migrations/0001_init.sql` (PostgreSQL 16+, ทดสอบ constraint แล้ว)

ผู้ใช้

`users` · **`user_identities`** · **`faculties`**

ผู้ใช้ 1 คน login ได้หลายช่องทาง · คณะอัปเดตจาก CUNEX ทุกครั้งที่ login

โพล

`polls` · **`poll_audience_rules`** · **`poll_reviews`**

state machine · ขอบเขตผู้ตอบ = OR ของ (คณะ AND ปีเข้า) · ตีกลับต้องมีเหตุผล

เนื้อหา

`poll_elements` · **`element_options`** · **`images`**

element 9 ชนิดอยู่ตารางเดียว (`config JSONB`) · trigger ล็อกเมื่อโพลเปิด · สลับลำดับได้ใน transaction เดียว

คำตอบ

`ballots` · **`ballot_choices`** · **`ballot_text_answers`**

ไม่มี user_id · composite FK กันส่งตัวเลือกข้ามคำถามหรือข้ามโพล · index สำหรับ GROUP BY

ระบบ

`notification_batches` · **`audit_logs`**

กันส่ง notification ซ้ำ · audit เฉพาะ action ของ admin และ lifecycle

ไม่อยู่ใน Postgres

Session · OTP

เก็บใน Redis แบบมี TTL หมดอายุเองโดยไม่ต้อง cleanup

สิ่งที่ DB บังคับเอง (ไม่พึ่ง API)

ล็อกโครงสร้าง

แก้คำถามหรือตัวเลือกไม่ได้เมื่อโพลไม่ได้อยู่ใน draft/rejected

1 คน 1 ballot

`UNIQUE(poll_id, voter_key)`

แม้ request จะเข้ามาพร้อมกัน

ไม่ข้ามโพล

composite FK: ตัวเลือก ∈ คำถาม ∈ โพลเดียวกับ ballot

10 สมมติฐานและคำถามที่ยังเปิดอยู่

สิ่งที่ต้องยืนยันก่อนเริ่ม implement

ต้องแก้ดีไซน์

- "เฉพาะคณะ/**สาขา**" → "เฉพาะคณะ/**ปีเข้า**" เพราะ CUNEX ไม่ส่งข้อมูลสาขา
- ป๊อปอัป "เลือกบัญชีทดลอง" เปิดได้เฉพาะ dev/staging

สมมติฐาน (ยังไม่ได้ยืนยัน)

- ระบบเป็นเว็บแยกที่เข้าได้ทั้งจาก CUNEX และเปิดตรง "หน้ารวม" หมายถึงหน้ารวมโพลในเว็บเรา
- ตัวเลขโหลดในหัวข้อ 01 เป็นค่าประมาณ

คำถามที่ต้องหาคำตอบ

คำถาม	ถามใคร	กระทบอะไร
มีข้อกำหนดว่าข้อมูลต้องอยู่ในไทย หรือต้อง deploy บน infra ของจุฬาฯ ไหม	สำนัก IT / DPO	ที่ deploy (architecture ไม่ต้องเปลี่ยน)
บัญชีบุคลากร/อาจารย์เรียก `/profile` แล้วได้อะไร	ทีม CUNEX	วิธีให้ Admin ที่ไม่ใช่นิสิต login
CUNEX ส่งชื่อผู้ใช้ได้ไหม (`/profile` ไม่มีชื่อ)	ทีม CUNEX	หน้า admin และหน้าโพลแสดงชื่อเจ้าของไม่ได้
โควตาและ rate limit ของ notification API	ทีม CUNEX	ขนาด batch และเวลาที่ใช้ส่งทั้งมหาวิทยาลัย
อายุของ CUNEX token	ทีม CUNEX	ความเสี่ยงที่ token ในลิงก์หมดอายุก่อนผู้ใช้เปิด
ค่า k สำหรับ k-anonymity	ทีม / DPO	ความละเอียดของผลแยกกลุ่มที่แสดงได้

ขั้นต่อไป

1. **ออกแบบ API**
   endpoint และ zod schema ของ element ทั้ง 9 ชนิด
2. **ตั้งโปรเจกต์**
   NestJS + migration จาก `db/migrations/0001_init.sql`
3. **Auth กับ CUNEX sandbox**
   ทำเป็นอย่างแรก เพราะทุกส่วนขึ้นกับมัน
4. **Load test**
   สถานการณ์ส่ง push ทั้งมหาวิทยาลัยก่อนเปิดใช้งานจริง